ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФГАОУ ВО НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет компьютерных наук
Образовательная программа «Прикладная математика и информатика»

Отчет о командном программном проекте на тему:

Прогнозирование динамики графа платежей на основе открытых платежных данных

(промежуточный, этап 1)

Выполнили студенты:

группы #БПМИ233, 3 курса	Кречетов Роман Константинович
группы #БПМИ233, 3 курса	Романюков Егор Сергеевич
группы #БПМИ2310, 3 курса	Шеин Илья Антонович

Принял руководитель проекта:

Филатов Дмитрий Станиславович
Исполнительный директор по исследованию данных
ПАО "Сбербанк"

Москва 2026

Содержание

Аннотация **4**

Список ключевых слов **4**

Введение **4**

Обзор литературы **4**

Обзор библиотек 4

Обзор работ про биткоин 4

Обзор моделей для прогнозирования 4

1 Задача **4**

1.1 Датасет 4

1.2 Постановка задачи 4

1.3 Семплирование негативных рёбер 5

1.4 Стратегия формирования кандидатов 6

1.5 Метрики качества 6

1.6 Признаки 7

2 Модели **9**

2.1 Базовые решения 9

2.2 EAGLE 9

2.3 GraphMixer 11

2.4 GLFormer 12

2.5 DyGFormer 14

3 Эксперименты **15**

3.1 Результаты экспериментов 15

4 Фреймворк **16**

5 CUDA **16**

Список литературы **17**

A Пример секции аппендикса **18**

B Описание применения генеративных моделей **18**

Аннотация

Список ключевых слов

Введение

Обзор литературы

1 Задача

1.1 Датасет

1.2 Постановка задачи

Рассматривается задача предсказания временных связей (temporal link prediction) в динамическом направленном графе [2, 3]. Динамический граф представляется как поток событий (stream graph) $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, где $\mathcal{V}$ — конечное множество узлов, а $\mathcal{E} = \{(s_i, d_i, t_i)\}_{i=1}^{|\mathcal{E}|}$ — хронологически упорядоченное множество направленных рёбер с временными метками: $t_1 \leq t_2 \leq \cdots \leq t_{|\mathcal{E}|}$. Каждое ребро $(s, d, t) \in \mathcal{E}$ означает взаимодействие от узла-источника $s \in \mathcal{V}$ к узлу-назначению $d \in \mathcal{V}$ в момент времени $t \in \mathbb{R}_+$.

Задача состоит в следующем: дан граф $\mathcal{G}_{<t} = (\mathcal{V}, \{(s_i, d_i, t_i) \in \mathcal{E} \mid t_i < t\})$, содержащий все взаимодействия, предшествующие моменту времени t; требуется оценить вероятность $P(s, d, t \mid \mathcal{G}_{<t})$ появления нового ребра (s, d, t) между заданной парой узлов (s, d).

Для обучения и оценки качества модели рёбра разбиваются на три непересекающихся непрерывных временных интервала. Пусть хронологически упорядоченные рёбра пронумерованы от 1 до $|\mathcal{E}|$. Тогда:

- **Обучающая выборка** $\mathcal{E}_{\text{train}}$: первые 70% рёбер по временному порядку, то есть $\mathcal{E}_{\text{train}} = \{(s_i, d_i, t_i) \mid i \leq \lfloor 0.7 \cdot |\mathcal{E}| \rfloor\}$.
- **Валидационная выборка** $\mathcal{E}_{\text{val}}$: следующие 15% рёбер.
- **Тестовая выборка** $\mathcal{E}_{\text{test}}$: последние 15% рёбер.

Такое разбиение по временному принципу (temporal split) исключает любую утечку данных из будущего в прошлое [4]: модель в момент оценки на тестовой выборке имеет доступ только к рёбрам из $\mathcal{E}_{\text{train}}$ и $\mathcal{E}_{\text{val}}$.

Помимо транзакционных рёбер, узлы и рёбра могут быть снабжены атрибутными признаками. В рамках данной работы признаки ребра кодируют объём транзакции в единицах биткоина. Функция узловых признаков $\mathbf{x}_v(t) \in \mathbb{R}^{d_v}$ представляет статические или динамические свойства узла v, вычисляемые на основе $\mathcal{E}_{\text{train}}$ (см. раздел о признаках). Таким образом, задача предсказания связей формально сводится к задаче бинарной классификации пар узлов (s, d) при заданном моменте времени t: модель должна отличить реальное ребро от заданного множества отрицательных примеров [5].

1.3 Семплирование негативных рёбер

При оценке модели на задаче предсказания связей для каждого истинного (положительного) ребра $(s, d, t) \in \mathcal{E}_{\text{eval}}$ необходимо сформировать набор отрицательных рёбер — несуществующих взаимодействий, которые модель должна отличить от реального. Качество и репрезентативность этого набора существенно влияют на оценки метрик [4].

Различают три основных стратегии семплирования негативных рёбер:

1 **Случайное семплирование (random negatives, RND).** Для каждого позитивного ребра (s, d, t) случайным образом сэмплируется узел $d' \in \mathcal{V}$, $d' \neq d$, и формируется отрицательный пример (s, d', t). Аналогично можно фиксировать d и случайным образом выбирать источник s'. Данная стратегия является наиболее распространённой, однако считается чрезмерно простой: большинство случайных пар узлов никогда не взаимодействовали, и модель может легко их отсеять.

2 **Исторически обоснованные негативы (historical negatives, HIST).** Отрицательные примеры выбираются из пар узлов, которые взаимодействовали в прошлом (до момента t), но не являются истинным ребром в данный момент. Такая стратегия существенно усложняет задачу, поскольку модель должна отличать реальное взаимодействие от схожих по структуре исторических.

3 **Индуктивные негативы (inductive negatives, IND).** Отрицательные примеры включают пары узлов, хотя бы один из которых не встречался в обучающей выборке. Данная стратегия проверяет способность модели обобщаться на новые (unseen) узлы, что особенно важно в индуктивном сценарии.

В данной работе используется стратегия случайного семплирования негативных рёбер с фиксированным узлом-источником: для каждого позитивного ребра (s, d, t) случайным образом выбирается один отрицательный узел назначения d', формируя пару (s, d', t).

1.4 Стратегия формирования кандидатов

При оценке моделей по метрикам ранжирования для каждого ребра из оценочной выборки формируется список кандидатов, состоящий из истинного ребра и N_{neg} отрицательных. Модель ранжирует кандидатов по предсказанной вероятности взаимодействия, и оценивается позиция истинного ребра в этом ранжировании.

Различают два основных подхода к формированию кандидатов:

- **Полный список кандидатов (full candidate set).** В качестве кандидатов рассматриваются все узлы $\mathcal{V}$, кроме истинного назначения. Метрики, вычисленные на полном списке кандидатов, отражают реальное качество ранжирования, однако вычислительно затратны для больших графов ($|\mathcal{V}|$ может составлять миллионы узлов).

- **Выборочный список кандидатов (sampled candidate set).** Для каждого позитивного ребра (s, d, t) случайным образом выбирается фиксированное число отрицательных примеров N_{neg}, которые вместе с истинным ребром образуют список кандидатов. Оценка производится на этом ограниченном списке. Данная схема позволяет эффективно вычислять метрики при большом числе узлов и широко используется в литературе [4].

В данной работе применяется схема с выборочным списком кандидатов: каждый позитивный пример оценивается в сравнении с одним случайным негативным примером (то есть $N_{\text{neg}} = 1$). В этом случае задача ранжирования сводится к бинарной классификации в паре (позитивный, негативный), а метрика MRR принимает значения $\{0.5, 1.0\}$ в зависимости от того, был ли позитивный пример поставлен выше негативного.

1.5 Метрики качества

Для оценки качества предсказания связей используются метрики ранжирования, стандартные для задач link prediction [1]. Пусть $\mathcal{E}_{\text{eval}}$ — оценочная выборка, а для каждого ребра $e_i = (s_i, d_i, t_i) \in \mathcal{E}_{\text{eval}}$ формируется список кандидатов $\mathcal{C}_i$, включающий истинное ребро e_i и набор отрицательных примеров. Модель присваивает каждому кандидату скоровую функцию $f(s, d, t)$, на основании которой кандидаты ранжируются в порядке убывания. Ранг истинного ребра обозначается r_i.

Mean Reciprocal Rank (MRR) — среднее значение обратного ранга истинного ребра

по всем тестовым примерам:

$$\mathrm{MRR} = \frac{1}{|\mathcal{E}_{\mathrm{eval}}|} \sum_{i=1}^{|\mathcal{E}_{\mathrm{eval}}|} \frac{1}{r_i}.$$

MRR принимает значения в $[0, 1]$; значение, близкое к 1, означает, что модель в среднем помещает истинное ребро на первое место в ранжировании.

Hits@K (H@K) — доля тестовых примеров, для которых истинное ребро оказалось в топ-K среди кандидатов:

$$\mathrm{H@K} = \frac{1}{|\mathcal{E}_{\mathrm{eval}}|} \sum_{i=1}^{|\mathcal{E}_{\mathrm{eval}}|} \mathbf{1}[r_i \leq K].$$

В данной работе используются $K = 1$ и $K = 10$, то есть H@1 (точность при одном ответе) и H@10 (охват в топ-10 кандидатах). H@1 совпадает с долей случаев, когда модель однозначно выбирает правильное ребро как наиболее вероятное.

Поскольку в данной работе для каждого позитивного ребра сэмплируется ровно один негативный кандидат ($|\mathcal{C}_i| = 2$), ранг r_i принимает значение 1 (если модель поставила позитивный пример выше негативного) или 2 (иначе). Следовательно, $\mathrm{MRR} \in \{0.5, 1.0\}$ для каждого примера в отдельности, а агрегированное MRR интерпретируется как доля правильно ранжированных пар.

1.6 Признаки

В рамках экспериментального протокола, основанного на принципах предотвращения утечки данных (data leakage), набор признаков, характеризующих как отдельные узлы, так и пары узлов (рёбра), формируется исключительно на основе обучающей выборки (train set), представляющей собой первые 70% хронологически упорядоченных рёбер внутри выделенного временного периода (period). Это гарантирует, что ни валидационная, ни тестовая выборки не вносят вклада в построение статических или динамических характеристик модели. Формально, пусть $\mathcal{E}_{\mathrm{train}}$ — множество рёбер в обучающем периоде, а $\mathcal{V}_{\mathrm{train}} = \{v \mid \exists e \in \mathcal{E}_{\mathrm{train}} : v \in e\}$ — множество уникальных узлов, встречающихся в этих рёбрах. Все признаки вычисляются на основе $\mathcal{E}_{\mathrm{train}}$ и $\mathcal{V}_{\mathrm{train}}$.

Каждый узел $v \in \mathcal{V}_{\mathrm{train}}$ характеризуется вектором из 15 числовых признаков, отражающих его структурные, временные и транзакционные свойства в рамках графа потока транзакций (stream graph). Для узла v эти признаки включают:

- **Степенные характеристики**: логарифм входящей степени ($\log \mathrm{in_degree}(v)$) и логарифм исходящей степени ($\log \mathrm{out_degree}(v)$), где степень определяется по количе-

ству уникальных рёбер в $\mathcal{E}_{\text{train}}$, а также отношение входящей степени к исходящей ($\text{in_out_ratio}(v)$) для оценки преобладающей роли узла как отправителя или получателя.

- **Характеристики уникальных контрагентов**: логарифм количества уникальных входящих контрагентов ($\log \text{unique_in_cp}(v)$) и исходящих контрагентов ($\log \text{unique_out_cp}($ что позволяет оценить разнообразие взаимодействий узла.

- **Транзакционные объемы**: логарифм суммарного входящего объема в биткоинах ($\log \text{total_btc_in}(v)$), логарифм суммарного исходящего объема ($\log \text{total_btc_out}(v)$), а также их усредненные значения по транзакциям ($\log \text{avg_btc_in}(v)$, $\log \text{avg_btc_out}(v)$).

- **Временные паттерны активности**: метрика недавности активности ($\text{recency}(v)$), определяемая как временной интервал между последним участием узла в ребре и концом обучающего периода; размах активности ($\text{activity_span}(v)$) как разность между максимальным и минимальным временными штампами участия узла; логарифм интенсивности событий ($\log \text{event_rate}(v)$), вычисляемый как отношение количества рёбер узла к его размаху активности; и коэффициент кластеризации событий во времени ($\text{burstiness}(v)$).

- **Энтропия контрагентов**: энтропия распределения исходящих контрагентов (out_counterpa и входящих контрагентов ($\text{in_counterparty_entropy}(v)$), характеризующая степень предсказуемости или разнообразия партнеров узла.

Для узлов, не входящих в $\mathcal{V}_{\text{train}}$, все 15 признаков полагаются равными нулю.

Для каждой предсказываемой пары (s, d), где s — узел-источник (source), d — узел-назначение (destination), формируется составной признак, объединяющий характеристики узлов. Результирующий вектор признаков для ребра имеет размерность 30 и конкатенирует:

1 15 признаков узла-источника s;

2 15 признаков узла-назначения d.

Для пар (s, d), где хотя бы один узел отсутствует в $\mathcal{V}_{\text{train}}$, все 30 признаков принимают нулевое значение, что соответствует нулевому вектору признаков для такого ребра. Данный набор признаков обеспечивает детерминированное и воспроизводимое представление индивидуальных свойств узлов, строго в рамках границ обучающих данных, и используется в качестве входных характеристик для моделей, выполняющих предсказание связей.

2 Модели

2.1 Базовые решения

2.2 EAGLE

Предлагаемая модель EAGLE (Efficient and Effective GrAph modEL for temporal Link prEdiction) представляет собой легковесную архитектуру, предназначенную для решения задачи предсказания временных связей в динамических графах. Основной принцип, заложенный в основу EAGLE, заключается в раздельном и явном учете двух ортогональных, но взаимодополняющих аспектов, значимых для прогнозирования взаимодействий: краткосрочной темпоральной активности узла, отражающей его непосредственные предпочтения, и долгосрочных глобальных структурных паттернов, определяющих его позицию в сети. В отличие от существующих темпоральных графовых нейронных сетей, использующих сложные механизмы, такие как многослойные трансформеры, рекуррентные блоки или модули памяти, EAGLE достигает высокой эффективности за счет простоты своих компонентов, агрегируя информацию лишь из ограниченного набора наиболее релевантных узлов. Архитектура EAGLE состоит из трех ключевых компонентов: модуля, учитывающего фактор времени (time-aware module), модуля, учитывающего структурную информацию (structure-aware module), и адаптивного механизма их объединения.

Модуль, учитывающий фактор времени, фокусируется на захвате краткосрочных темпоральных трендов. Для заданного узла v в момент времени t из всего множества его исторических соседей $N_v(t)$ отбираются k_r наиболее поздних по времени взаимодействия соседей, формируя множество $N_v(t, k_r)$. Такой подход обоснован экспериментально: недавние взаимодействия являются наиболее сильными индикаторами будущих связей. Далее, для узла v вычисляется его временное представление $\mathbf{h}_v^{\text{ta}}(t)$ путем усреднения конкатенации признаков самих соседей $\mathbf{x}_{v_i}(t)$ и признаков ребер $\mathbf{e}_{v,v_i}(\tau_i)$, связывающих их с узлом v:

$$\mathbf{h}_v^{\text{ta}}(t) = \frac{1}{k_r} \sum_{(v_i, \mathbf{e}_{v,v_i}(\tau_i), \tau_i) \in N_v(t,k_r)} [\mathbf{x}_{v_i}(t), \mathbf{e}_{v,v_i}(\tau_i)].$$

Для пары узлов (v, u) вероятность наличия связи на основе временного модуля $s_{v,u}^{\text{ta}}(t)$ вычисляется с помощью двухслойного перцептрона (MLP), на вход которого подается конкатенация их временных представлений:

$$s_{v,u}^{\text{ta}}(t) = \sigma(\mathbf{W}_2 \cdot \text{ReLU}(\mathbf{W}_1[\mathbf{h}_v^{\text{ta}}(t), \mathbf{h}_u^{\text{ta}}(t)] + \mathbf{b}_1) + \mathbf{b}_2),$$

где σ — сигмоидная функция, а $\mathbf{W}_1, \mathbf{W}_2, \mathbf{b}_1, \mathbf{b}_2$ — обучаемые параметры.

Модуль, учитывающий структурную информацию, предназначен для захвата долгосрочных глобальных зависимостей. Его работа основана на алгоритме темпорального персонализированного ранжирования PageRank (T-PPR), который обобщает классический PPR на динамические графы, присваивая больший вес более поздним взаимодействиям при вычислении вероятностей перехода. Для узла v вычисляется вектор $\boldsymbol{\pi}_v(t)$, где каждый элемент $\boldsymbol{\pi}_v(t)[u]$ представляет собой T-PPR-оценку влияния узла u на узел v в момент времени t. Для каждого узла отбирается множество из k_s узлов, имеющих наибольшие T-PPR-оценки. Структурная связь между узлами v и u оценивается путем суммирования произведений их T-PPR-оценок для общих влиятельных узлов:

$$s_{v,u}^{\mathrm{sa}}(t) = \sum_{v_i \in N_v(\boldsymbol{\pi}_v(t), k_s) \cap N_u(\boldsymbol{\pi}_u(t), k_s)} \boldsymbol{\pi}_v(t)[v_i] \cdot \boldsymbol{\pi}_u(t)[v_i].$$

Полученная величина $s_{v,u}^{\mathrm{sa}}(t)$ является детерминированной и не требует обучения, что делает этот модуль исключительно легковесным.

Финальный гибридный модуль объединяет выходы временного и структурного компонентов, используя адаптивный механизм взвешивания. Основная идея заключается в том, чтобы динамически регулировать вклад временного модуля в зависимости от того, насколько недавними были взаимодействия узлов. Для этого для узла v вычисляется средний интервал времени $\bar{t}_v$ до его k_r последних соседей. Адаптивный вес для временной оценки задается как $\exp(-\bar{t}_v)$. Итоговая гибридная оценка для пары (v, u) определяется как:

$$s_{v,u}^{\mathrm{hybrid}}(t) = \lambda \cdot (\exp(-\bar{t}_v) + \exp(-\bar{t}_u)) \cdot s_{v,u}^{\mathrm{ta}}(t) + s_{v,u}^{\mathrm{sa}}(t),$$

где λ — гиперпараметр, регулирующий общий баланс между двумя источниками информации. Такая схема позволяет модели автоматически отдавать приоритет недавним взаимодействиям для активно меняющихся узлов и полагаться на глобальную структуру для узлов с редкой или давней активностью. Вся архитектура EAGLE, благодаря использованию простых операций усреднения, MLP ограниченной глубины и эффективному обновлению T-PPR, демонстрирует линейную или почти линейную вычислительную сложность относительно числа новых событий и предсказываемых пар узлов.

2.3 GraphMixer

Модель GraphMixer (Cong et al., 2023) представляет собой архитектуру для прогнозирования связей в темпоральных графах, разработанную с целью минимизации концептуальной и технической сложности. В отличие от преобладающих подходов, использующих рекуррентные нейронные сети (RNN) или механизмы самовнимания (SAM), GraphMixer построена исключительно на основе многослойных перцептронов (MLP) и операции пулинга соседей, что позволяет достичь конкурентоспособной производительности без усложнения архитектуры. Модель состоит из трёх основных функциональных модулей: кодировщика связей, кодировщика узлов и классификатора связей.

Кодировщик связей (link-encoder) предназначен для обобщения информации о темпоральных связях, ассоциированных с каждым узлом. Для узла v_i его темпоральная история представляет собой упорядоченный по времени набор взаимодействий с другими узлами. На вход кодировщика подаются данные о K наиболее поздних взаимодействиях, где K — гиперпараметр, зависящий от набора данных. Каждое взаимодействие в момент времени t кодируется путём конкатенации вектора признаков связи $\mathbf{x}_{ij}^{\text{link}}(t)$ с результатом применения фиксированной функции временного кодирования $\cos(t\boldsymbol{\omega})$ к относительному времени $(t_0 - t)$, где t_0 — целевой момент прогнозирования. Вектор $\boldsymbol{\omega} = \{\alpha^{-(i-1)/\beta}\}_{i=1}^{d}$ содержит фиксированные, не обучаемые в процессе тренировки частотные компоненты, где параметры α и β выбираются таким образом, чтобы гарантировать различимость всех временных меток в заданном диапазоне. Для унификации длины входных последовательностей используется дополнение нулями (zero-padding) до фиксированной длины K, что позволяет модели учитывать частоту взаимодействий узла. Полученная матрица $\mathbf{T}_i(t_0)$ размерности $K \times d_{\text{input}}$ обрабатывается однослойным MLP-Mixer (Tolstikhin et al., 2021). Архитектура MLP-Mixer последовательно применяет два преобразования: сначала токен-микшер (token-mixer), который позволяет обмениваться информацией между различными взаимодействиями (токенами) в последовательности, затем канальный микшер (channel-mixer), обрабатывающий признаки независимо для каждого токена. Оба микшера используют операции линейных преобразований, функции активации GeLU и нормализацию слоя (LayerNorm). Итоговым представлением темпоральной информации узла v_i является вектор $\mathbf{t}_i(t_0)$, получаемый усреднением (mean-pooling) выходной матрицы канального микшера.

Кодировщик узлов (node-encoder) предназначен для агрегации статической информации об узле и его ближайшем окружении. Представление узла формируется на основе его собственных признаков $\mathbf{x}_i^{\text{node}}$ и признаков его 1-соседей, с которыми были зафиксированы

взаимодействия за временной интервал $[t_0 - T, t_0]$, где T — гиперпараметр. Итоговый вектор $\mathbf{s}_i(t_0)$ вычисляется как сумма собственных признаков узла и среднего значения признаков его соседей: $\mathbf{s}_i(t_0) = \mathbf{x}_i^{\text{node}} + \text{Mean}\{\mathbf{x}_j^{\text{node}} \mid v_j \in \mathcal{N}(v_i; t_0 - T, t_0)\}$. В случае отсутствия атрибутивных признаков узлов в наборе данных используются one-hot векторы для идентификации узлов.

Классификатор связей (link classifier) является двухслойным MLP, который на основе объединённых представлений пары узлов вычисляет вероятность существования связи между ними в момент времени t_0. Для узлов v_i и v_j итоговые представления $\mathbf{h}_i(t_0) = [\mathbf{s}_i(t_0) \| \mathbf{t}_i(t_0)]$ и $\mathbf{h}_j(t_0) = [\mathbf{s}_j(t_0) \| \mathbf{t}_j(t_0)]$ конкатенируются и подаются на вход классификатора: $p_{ij} = \text{MLP}([\mathbf{h}_i(t_0) \| \mathbf{h}_j(t_0)])$.

Важной архитектурной особенностью GraphMixer является рассмотрение темпорального графа как неориентированного. Это позволяет кодировщику узлов извлекать информацию о частоте взаимных связей между узлами в недавнем прошлом, что является значимым признаком для прогнозирования. Выборка ограничивается только 1-соседями с наиболее поздними временными метками, что снижает зашумленность входных данных и уменьшает вычислительную сложность по сравнению с методами, использующими многошаговую или равномерную выборку соседей.

2.4 GLFormer

Предлагаемая модель GLFormer (Global-Lens Transformer) представляет собой архитектуру для обучения на динамических графах, которая отказывается от классического механизма self-attention в пользу более эффективного адаптивного смешения токенов. Основная идея заключается в том, что успех Transformer-подобных архитектур в значительной степени обусловлен их общей структурой, а не самим механизмом внимания. В связи с этим GLFormer строит процесс обработки последовательностей взаимодействий узлов, комбинируя два ключевых компонента: адаптивный модуль агрегации токенов и иерархический механизм агрегации для расширения временного рецептивного поля.

Пусть для заданного взаимодействия $e = (u, v, t)$ получены начальные эмбеддинги узлов с помощью некоторой функции внедрения f_E, характерной для существующих методов обучения на темпоральных графах: $\mathbf{X}_u = f_E(\mathcal{G}_{t-}, u)$. Входными данными для GLFormer служит последовательность эмбеддингов соседей узла u: $\mathbf{I}_u = [\mathbf{X}_{u_1}, \ldots, \mathbf{X}_{u_N}] \in \mathbb{R}^{N \times d}$, где N — количество рассматриваемых соседей, а d — размерность эмбеддинга. Основу архитектуры составляют два последовательно применяемых блока: смеситель токенов (token mixer), отвечающий за агрегацию информации между позициями последовательности, и канальный

смеситель (channel mixer), моделирующий взаимосвязи между различными признаковыми каналами. Каждый блок дополнен остаточным соединением для стабилизации обучения и интеграции исходных данных.

Адаптивный модуль агрегации токенов выполняет локальное взвешенное суммирование по M наиболее недавним соседям. Для каждого соседа u_i, взаимодействовавшего в момент времени t_i, его обновленное представление $H_{i,:}$ вычисляется как взвешенная сумма эмбеддингов предшествующих соседей $\mathbf{I}_{i-p,:}$, где $p = 0, \ldots, M-1$. Веса агрегации α_p^i определяются двумя факторами: обучаемыми параметрами w_p, отражающими важность позиционного порядка, и временный́ фактором θ_p^i, основанным на относительных разностях времен взаимодействий. Итоговый вес задается комбинацией с обучаемым параметром β: $\alpha_p^i = \beta w_p + (1-\beta)\theta_p^i$. Временной фактор вычисляется как softmax от отрицательных значений временных интервалов: $\theta_p^i = \frac{\exp(-(t_i - t_{i-p}))}{\sum_{q=0}^{M-1}\exp(-(t_i - t_{i-q}))}$. Такой подход позволяет модели динамически адаптировать агрегацию с учетом как порядка следования взаимодействий, так и их временной близости.

Для захвата долгосрочных зависимостей, выходящих за пределы локального окна, в GLFormer реализован иерархический механизм агрегации. Идея заключается в том, чтобы с увеличением глубины слоя расширять рецептивное поле за счет увеличения шага и количества учитываемых соседей. На слое l смеситель токенов использует ядро размера K_l, определяемое множеством смещений $\mathcal{R}_l = \{p \in \mathbb{Z} \mid s^{l-1} \leq p \leq s^l\}$, где s — параметр, контролирующий экспоненциальный рост рецептивного поля. Агрегация на этом уровне описывается выражением: $\mathbf{H}_{i,:}^{(l)} = \sum_{p\in\mathcal{R}_l}(\alpha_p^{(i)})^{(l)}\mathbf{H}_{\mathrm{TA},i-p}^{(l-1)}$, где $\mathbf{H}_{\mathrm{TA},i-p}^{(l-1)}$ — выход предыдущего слоя после обработки канальным смесителем. Таким образом, каждый последующий слой оперирует все более широким временный́ контекстом, что позволяет эффективно моделировать как краткосрочную динамику, так и долгосрочные закономерности.

Канальный смеситель соответствует стандартной реализации из архитектуры Transformer. После модуля агрегации токенов выполняется проекция, остаточное соединение и нормализация слоя, после чего следует двухслойный feed-forward network с функцией активации GELU и вторым остаточным соединением. Объединяя L таких слоев, GLFormer получает итоговое представление последовательности $\mathbf{H}_{\mathrm{TA}}^{(L)}$. Финальное темпоральное представление узла u формируется с помощью операции среднего пулинга по всем позициям последовательности: $\mathbf{Z}_u = \frac{1}{N}\sum_{i=1}^{N}\mathbf{H}_{\mathrm{TA},i,:}^{(L)}$. Такая архитектура позволяет существенно снизить вычислительную сложность по сравнению со стандартным self-attention (с $O(N^2)$ до $O(\sum_{l=1}^{L} NK_l)$), сохраняя при этом способность к эффективному моделированию сложных временны́х зависимостей в динамических графах.

2.5 DyGFormer

Предлагаемая архитектура DyGFormer (Dynamic Graph Transformer) основана на применении трансформера для обработки последовательностей исторических взаимодействий узлов. Ключевой принцип заключается в том, чтобы для прогнозирования взаимодействия между узлами-источником u и узлом-назначением v в момент времени t использовать исключительно последовательности их первых соседей (first-hop interactions), предшествующих данному моменту. Такой подход сводит задачу обучения на динамическом графе к задаче анализа последовательностей.

Для каждого узла u формируется последовательность его исторических взаимодействий $\mathcal{S}_u^t = \{(u, u', t') | t' < t\} \cup \{(u', u, t') | t' < t\}$, аналогично $\mathcal{S}_v^t$ для узла v. Из этих последовательностей извлекаются четыре типа признаков: признаки соседей $\mathbf{X}_{u,N}^t \in \mathbb{R}^{|\mathcal{S}_u^t| \times d_N}$, признаки связей $\mathbf{X}_{u,E}^t \in \mathbb{R}^{|\mathcal{S}_u^t| \times d_E}$ и кодировка временных интервалов $\mathbf{X}_{u,T}^t \in \mathbb{R}^{|\mathcal{S}_u^t| \times d_T}$. Кодировка временных интервалов для интервала $\Delta t' = t - t'$ вычисляется с использованием обучаемых параметров $w_1, \ldots, w_{d_T}$ как $[\cos(w_1 \Delta t'), \sin(w_1 \Delta t'), \ldots, \cos(w_{d_T} \Delta t'), \sin(w_{d_T} \Delta t')]$.

Ключевым нововведением является схема кодирования совместной встречаемости соседей (Neighbor Co-occurrence Encoding, NCoE), предназначенная для моделирования корреляции между узлами u и v. Для каждого соседа в последовательностях $\mathcal{S}_u^t$ и $\mathcal{S}_v^t$ подсчитывается частота его появления в обеих последовательностях, формируя двумерный вектор. Совокупность таких векторов для всех соседей образует признаки совместной встречаемости $\mathbf{C}_u^t \in \mathbb{R}^{|\mathcal{S}_u^t| \times 2}$ и $\mathbf{C}_v^t \in \mathbb{R}^{|\mathcal{S}_v^t| \times 2}$. Далее, эти признаки преобразуются с помощью двухслойного персептрона $f(\cdot)$ с функцией активации ReLU: $\mathbf{X}_{u,C}^t = f(\mathbf{C}_u^t[:, 0]) + f(\mathbf{C}_u^t[:, 1]) \in \mathbb{R}^{|\mathcal{S}_u^t| \times d_C}$.

Для эффективной обработки длинных исторических последовательностей применяется техника патчинга (patching). Каждая последовательность признаков (например, $\mathbf{X}_{u,N}^t$) разбивается на непересекающиеся патчи размера P, содержащие P временно соседних взаимодействий. Патчи объединяются (flatten) в единые векторы, что позволяет сформировать матрицу патчей $\mathbf{M}_{u,N}^t \in \mathbb{R}^{l_u^t \times (d_N \cdot P)}$, где $l_u^t = \lceil |\mathcal{S}_u^t| / P \rceil$ — количество патчей. Размер патча P увеличивается с ростом длины последовательности, благодаря чему количество патчей l_u^t остается постоянным, а вычислительная сложность модели не зависит от исходной длины входа.

Далее все типы признаков для узлов u и v проецируются в единое пространство размерности d с помощью обучаемых весов $\mathbf{W}_*$ и смещений $\mathbf{b}_*$, после чего конкатенируются: $\mathbf{Z}_u^t = [\mathbf{Z}_{u,N}^t \| \mathbf{Z}_{u,E}^t \| \mathbf{Z}_{u,T}^t \| \mathbf{Z}_{u,C}^t] \in \mathbb{R}^{l_u^t \times 4d}$ и аналогично $\mathbf{Z}_v^t \in \mathbb{R}^{l_v^t \times 4d}$. Полученные представления объединяются в общую входную матрицу $\mathbf{Z}^t = [\mathbf{Z}_u^t; \mathbf{Z}_v^t] \in \mathbb{R}^{(l_u^t + l_v^t) \times 4d}$, которая подается на

вход кодировщика трансформера.

Кодировщик трансформера состоит из L слоев, каждый из которых включает блок многоголового самовнимания (Multi-head Self-Attention, MSA) и полносвязную сеть (Feed-Forward Network, FFN). В отличие от оригинальной архитектуры трансформера, в данной работе используется предварительная нормализация слоя (Pre-Layer Normalization) и функция активации GELU. Вычисление для l-го слоя осуществляется следующим образом:

$$\mathbf{O}^{t,l} = \mathrm{MSA}(\mathrm{LN}(\mathbf{Z}^{t,l-1})) + \mathbf{Z}^{t,l-1},$$
$$\mathbf{Z}^{t,l} = \mathrm{FFN}(\mathrm{LN}(\mathbf{O}^{t,l})) + \mathbf{O}^{t,l},$$

где $\mathrm{MSA}(\cdot)$ — многоголовое самовнимание, $\mathrm{FFN}(\cdot)$ — полносвязная сеть, $\mathrm{LN}(\cdot)$ — нормализация слоя, а $\mathbf{Z}^{t,0} = \mathbf{Z}^t$. Выход последнего слоя обозначается как $\mathbf{H}^t = \mathbf{Z}^{t,L} \in \mathbb{R}^{(l_u^t + l_v^t) \times 4d}$.

Финальные временные представления узлов u и v получаются путем усреднения соответствующих частей выходной матрицы $\mathbf{H}^t$ с последующим линейным преобразованием:

$$\mathbf{h}_u^t = \mathrm{MEAN}\left(\mathbf{H}^t[1 : l_u^t, :]\right) \mathbf{W}_{\mathrm{out}} + \mathbf{b}_{\mathrm{out}},$$
$$\mathbf{h}_v^t = \mathrm{MEAN}\left(\mathbf{H}^t[l_u^t + 1 : l_u^t + l_v^t, :]\right) \mathbf{W}_{\mathrm{out}} + \mathbf{b}_{\mathrm{out}},$$

где $\mathbf{W}_{\mathrm{out}} \in \mathbb{R}^{4d \times d_{\mathrm{out}}}$ и $\mathbf{b}_{\mathrm{out}} \in \mathbb{R}^{d_{\mathrm{out}}}$ — обучаемые параметры, d_{out} — размерность выходного представления. Полученные векторы $\mathbf{h}_u^t$ и $\mathbf{h}_v^t$ могут быть использованы для решения задач динамического прогнозирования связей и классификации узлов.

3 Эксперименты

3.1 Результаты экспериментов

Таблица 3.1: Эксперименты, сравнивающие различные модели

Модель	test MRR	val MRR	test H@1	test H@10
CN	0.64	0.70	0.55	0.81
AA	0.63	0.69	0.54	0.79
CN + MLP	0.62	0.69	0.53	0.79
Jaccard	0.55	0.63	0.45	0.75
RF	0.52	0.56	0.41	0.74
PA	0.50	0.53	0.41	0.69
Catboost	0.49	0.54	0.38	0.73
LogReg	0.35	0.36	0.26	0.52
EAGLE	0.46	0.53	0.35	0.67
DyGFormer	0.40	0.45		
GraphMixer	0.22	0.25	0.13	0.40
GLFormer	0.09	0.07	0.11	0.20

4 Фреймворк

5 CUDA

Список литературы

[1] Antoine Bordes, Nicolas Usunier, Alberto Garcia-Duran, Jason Weston и Oksana Yakhnenko. “Translating Embeddings for Modeling Multi-relational Data”. В: *Advances in Neural Information Processing Systems (NeurIPS)* (2013).

[2] Seyed Mehran Kazemi, Rishab Goel, Kshitij Jain, Ivan Kobyzev, Akshay Sethi, Peter Forsyth и Pascal Poupart. “Representation Learning for Dynamic Graphs: A Survey”. В: *Journal of Machine Learning Research* 21.70 (2020), с. 1—73.

[3] Antonio Longa, Veronica Lachi, Gabriele Santin, Monica Bianchini, Bruno Lepri, Pietro Lio, Franco Scarselli и Andrea Passerini. “Graph Neural Networks for Temporal Graphs: State of the Art, Open Challenges, and Opportunities”. В: *Transactions on Machine Learning Research*. 2023.

[4] Farimah Poursafaei, Shenyang Huang, Kellin Pelrine и Reihaneh Rabbany. “Towards Better Evaluation for Dynamic Link Prediction”. В: *Advances in Neural Information Processing Systems (NeurIPS)*. 2022.

[5] Da Xu, Chuanwei Ruan, Evren Korpeoglu, Sushant Kumar и Kannan Achan. “Inductive Representation Learning on Temporal Graphs”. В: *International Conference on Learning Representations (ICLR)*. 2020.

A Пример секции аппендикса

B Описание применения генеративных моделей

всё сами